

HALALAN REVAMP

Ayson, Aaron Eldrich
Elizarde, Samantha Coleen
Manzano, John Anthony

Client: University Computer Center

<https://cs191-ame.github.io/blog/>

CS 191 WFR1
Prof. Ligaya Leah Figueroa

Customer Statement of Requirements

Since the late 2000s, Halalan has served the UP community by providing an accessible and automated platform for online elections. Designed by the University of the Philippines Linux Users' Group (UnPLUG) for student elections within UP Diliman, it has since been used by other local universities in the Philippines including constituent universities within the UP system, and organizations such as the Land Bank of the Philippines Employees Association. Halalan is a free and open-source software currently being maintained by the University Computer Center (UCC) at UP Diliman. It can be accessed on GitHub via this link: <https://github.com/kadgabriel/halalan>.

Prior to the implementation of Halalan, elections were held on campus by the different colleges within the university with the help of a pool of volunteers. The tallying of votes was done traditionally, which was labor and resource intensive. Additionally, due to the manual nature of the elections, it was prone to human error. Halalan was created with the aim of addressing the issues that arose from the previous system of elections. By automating the election process, voting became more efficient, accessible, and secure.

The computerized voting system sped up many processes that were consuming a lot of time, effort, and resources, such as the checking of eligibility of voters, authentication of voters during the voting, processing of ballots, and tallying of votes. Many of these processes were made more efficient with the integration of other tools and software such as CSV files, sheets, databases, and email verification. These different resources helped streamline the processes before, during, and after the elections. By hosting the elections online and with the use of free software, it became considerably less costly.

The Halalan initiative made elections more convenient for users. Before Halalan and during the first few iterations of it, voters needed to cast their ballots in a specific location on campus or using designated machines at allocated times, which would disrupt scheduled classes, which could be inconvenient for many. With the updated software, the elections became accessible from any device. Users are able to cast their ballots remotely with the use of verifiable

credentials such as their registered UP email from the University's student database. The convenience this offered to users ultimately led to an increase in voter turnout for the elections. Apart from convenience, it is also accessible in the fact that it is open source, meaning it is free for others outside the university to use and adapt as they need. By openly providing an avenue to host computerized voting, this promotes secure and democratic elections, and empowers other institutions to do so as well.

The transformation to a digital platform also allowed for increased security, transparency, and efficient coordination between the different offices and stakeholders involved. The integration and sharing of databases made the authentication of users and ballots smoother. It minimized the margin for human error with the automated processes of counting the votes and the deliberate system functionalities established for the preservation of the integrity of the votes. The automation also made regular updates on the results readily available for anyone to access. These satisfied their motivation to prove that the results of a computerized election are valid and trustworthy.

The website is managed by UCC and designated college administrators. Going into the elections, UCC enters all the information regarding the election, including the parties and all the candidates running for the various positions. They then create different blocks corresponding to all the possible sets of election levels any student in the system can vote in. UCC coordinates with the different colleges in the university to collect and import the list of eligible voters in a formatted file. Information is pulled from the CRS database to encode the data received. Once all the relevant data is prepared and uploaded, the elections are ready to be hosted. For the duration of the election proper, the administrators lose access to edit anything on the website that may tamper with the results of the elections as one of the designed security protocols. The college admins are to host the elections for their respective colleges only, in accordance with the timeline set. The turnout and results of the election are released automatically.

Halalan has been successful in hosting university-wide elections throughout the UP system, other universities and organizations up to the present day, but several issues in the current implementation have been made known. Most notably, the programming language used

for the project is PHP version 8.0, and the CodeIgniter version 3 web framework. It is predicted that in the near future, support may be lost for these outdated versions of the programming language and web framework, and may therefore make the software more prone to security issues. Since the program was created back in the 2000s, it does not follow some of the modern conventions that developers adhere to presently, such as the Model-View-Controller (MVC) framework. Additionally, it was noted that during the tallying of results, users frequently checking for updates in the results page led to many queries on the database, which is resource-consuming. Furthermore, some interface improvements were suggested, such as making the voting on the user side compatible with mobile devices, and optimizing the admin interface with more advanced and intuitive tools. An effort was also made to include the CAPTCHA module for an extra layer of user verification, but the module cannot be run on certain versions of PHP. Lastly, the tradeoff for a voter being able to view their ballot after casting it is that, even with encryption, their vote could be traced back to them, making it a significant security risk.

In order to mitigate these security risks and preserve the well-established computerized elections, the Halalan website will be revamped with the latest version of PHP (version 8.5) and CodeIgniter (version 4.7). The program will generally follow the same flow of the current implementation on both the side of administrators and voters to ensure a smooth shift once it is deployed, but will feature changes to address the identified issues.

Requirements

- **Update existing codebase and framework to the most recent versions.** The Halalan website will be rewritten in PHP 8.5 and CodeIgniter 4.7, the most updated stable versions of the programming language and web framework, for reduced security risks, more advanced tools, and guaranteed support moving forward.
- **Application of Model-View-Controller framework.** Code segments will be rearranged so that they are under the appropriate part or file in the MVC framework.
- Implement upgrades such as better block voting systems and interface for admin users.
- Voting users select their chosen candidates with warning for invalid votes.

- **Integrate additional security features for authentication of voters.** The system currently uses a voter's UP email to authenticate their account. Additional measures such as a CAPTCHA can add an additional layer of authentication to increase security.
- **Develop a system for updating results.** Frequent updating of results and accessing the results page is heavy and resource intensive as it makes a lot of queries to the database. The current system gets the results every hour, exports it to a PDF file, and uploads this in the results page. The new implementation will seek out ways to improve this system in order to have more timely updates while minimizing the number of queries to the database. Also considered was a feature of withholding the final election results so that it may be released at one instance. This may involve disabling access to the results page upon the finalization of the results.
- **Focus on automated testing.**
- **Implement better encryption or alternative for returning ballot receipt.** The weakness of the computerized voting system is that in order for a voter's ballot receipt to be accessible every time they log into the system, the system must have a way to trace the voter's receipt back to them. In the implementation, possible alternative methods to this will be explored in order to maintain the same functionalities while limiting the security risks associated with it.

Preliminary UI Design

The existing interface of Halalan is designed to be lightweight and responsive. Figure 1 shows the voter login page for UP Diliman student elections. The login page originally displayed fields for the student number and password, but currently uses UP mail login instead. Additional login methods and displays should also be available for use outside of the university. The voting pages themselves are a series of checkboxes of available choices per position. For example in Figure 2, the abstain choice can be checked and submitted.

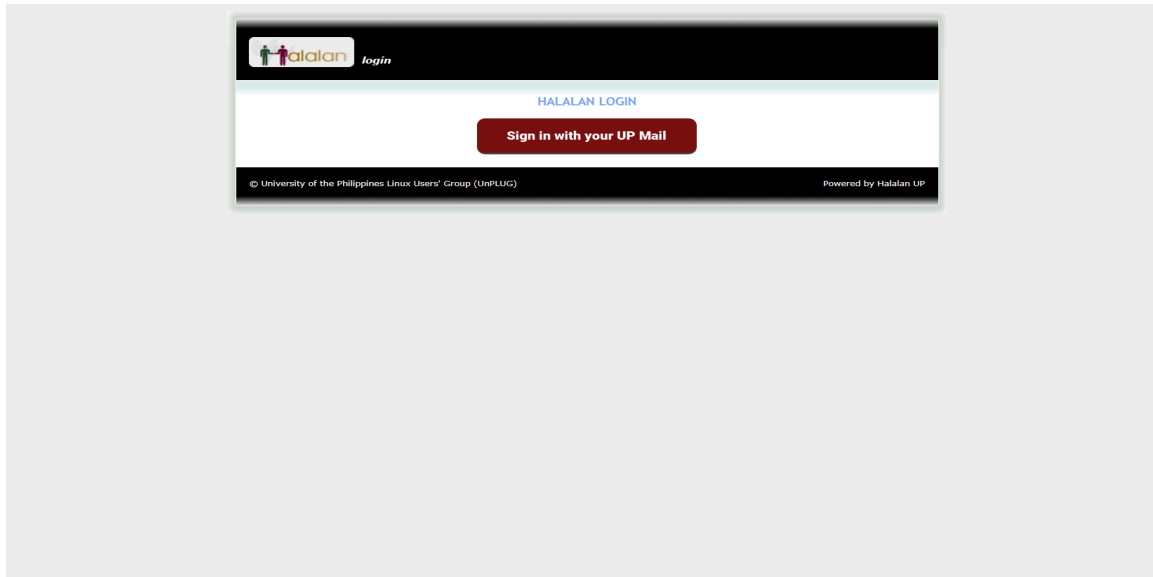


Figure 1. Halalan Voter Login

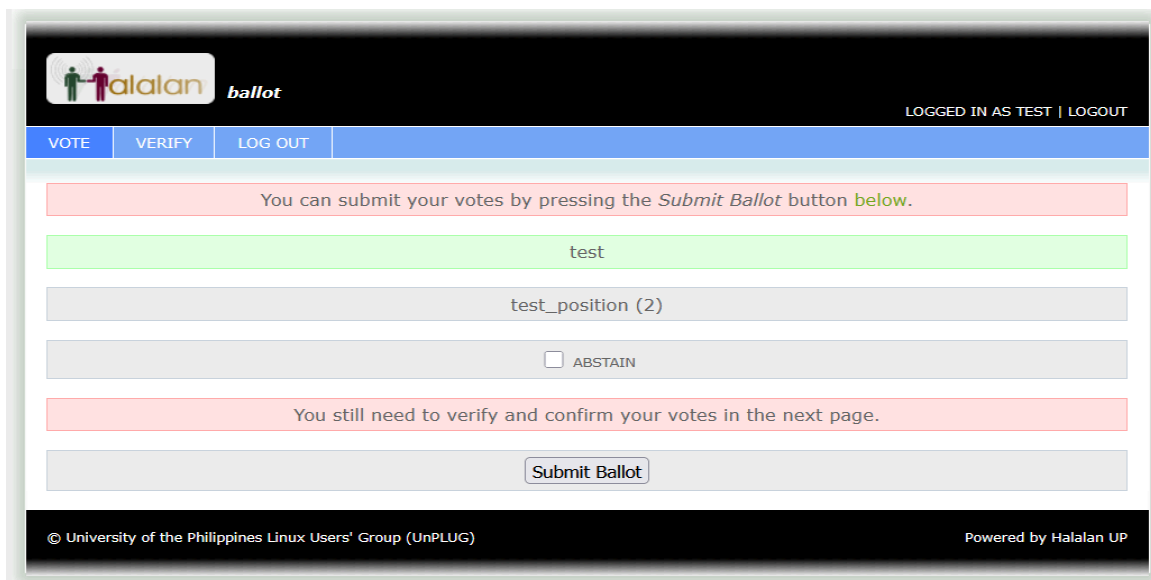


Figure 2. Voting page

Admin users land on the home page shown in Figure 3, from which they can navigate to CRUD modules such as Elections or Blocks. Additional features such as dragging and dropping (for example in images for candidates) may also be implemented. Figure 4 displays a sample error of editing ongoing elections which is not allowed by the system. Lastly, Figure 5 shows a statistics tab available for administrators in the login page.

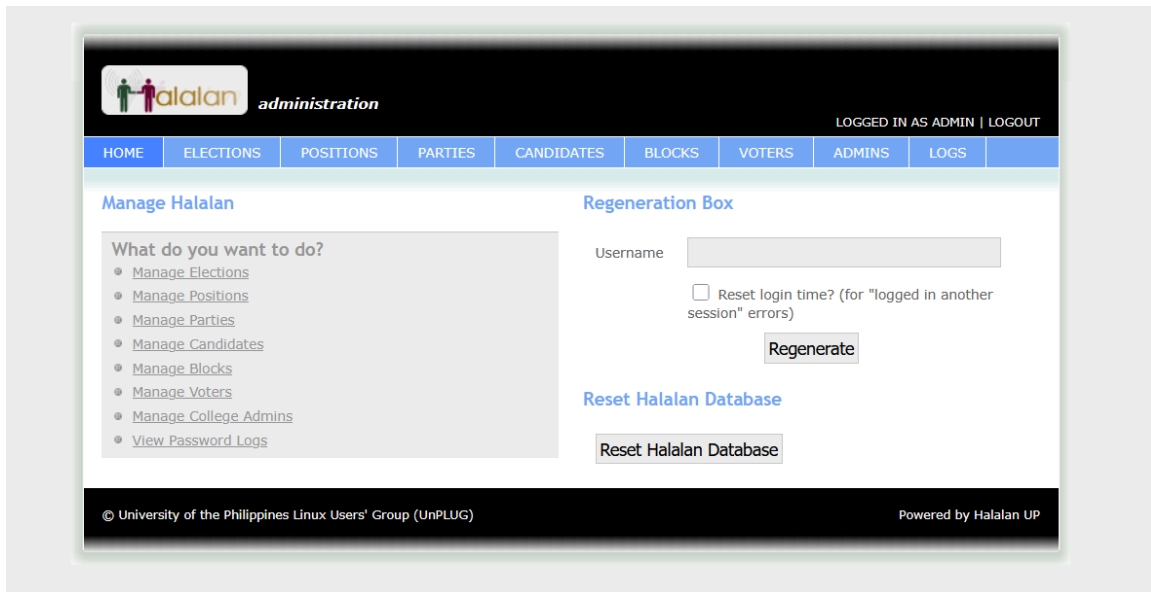


Figure 3. Halalan Admin Home Page

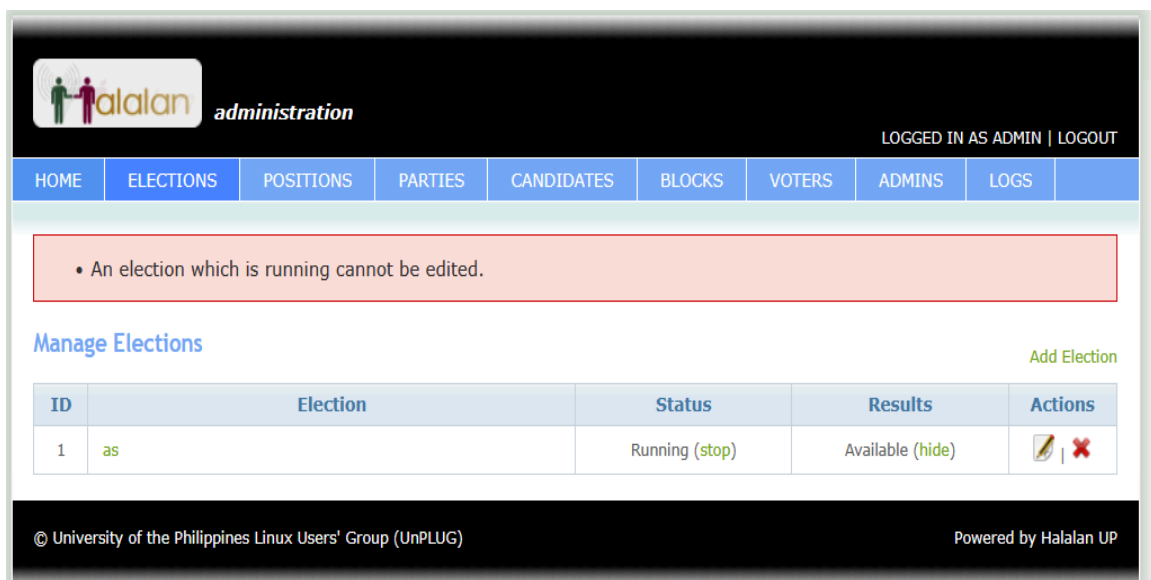


Figure 4. Manage Elections Tab

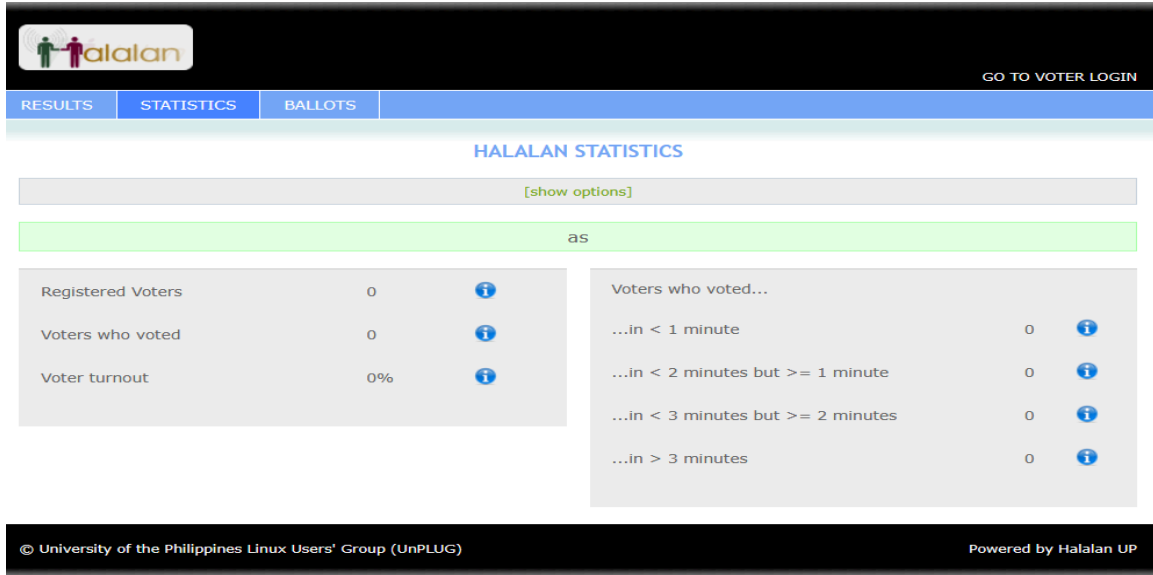


Figure 5. View Statistics Page

Plan of Work

Estimate cost and resources

We can estimate the load of this project to be 138.25 person-days (8 hours per day). We divide evenly among our three developers, which results in an estimated 47 person-days per person (rounded up for risk). The duration of this project is set to be until the end of the First Semester AY 2026-2027, with the developers working on the project part-time. Therefore, given the estimated project load, we conclude that this project is realizable within the timeframe. This is justified by the use of two estimation techniques: namely Function Point-based Estimation and Process-based Estimation.

Technique 1: Function Point-based Estimation

Information domain value		Count (est.)	Weighing Factor	FP
Number of external inputs	login, cast/confirm vote, add/edit	12	3	36

	election, position, party, candidate, block, import voters, generate/regenerate password			
Number of external outputs	results statistics, turnout report, voter-status export, results PDF	4	4	16
Number of external inquiries	view own vote, check voting status, search voter, view turnout	4	3	12
Number of internal logical files	Elections, Positions, Parties, Candidates, Blocks, Voters, Votes, Admin Users	8	7	56
Number of external interface files	CRS voter data, UP Mail SSO	2	5	10
Total FP				130

Table 1. FP-based Estimation Table

$$FP_{estimated} = FP\ count\ total \times [0.65 + 0.01 \times \sum(F_i)]$$

$$= 130 \times 1.00$$

$$= 130\ FP$$

$$Effort = 130\ FP \times \sim 10\ hrs/FP$$

$$= 1300\ hrs$$

Using FP-based estimation and an assumed productivity of 10 hours per FP, it has been deduced that the project will take around 1300 hours or 162.5 person-days. This translates to 54.17 person-days per person, which can be feasible within one academic semester.

Technique 2: Process-based Estimation

Activity	Engineering	Construction Release	Misc.	Total
----------	-------------	-------------------------	-------	-------

	Analysis	Design	Code	Test		
Communication and Planning	-	-	-	-	12	12
Authentication Module	1	2	4	2	-	9
Admin Modules	4	4	12	6	-	26
Voter Management System	2	2	6	3	-	14
Voting Module	3	4	5	4	-	14
Results and Turnout Reports	4	2	4	2	-	12
Security and Anonymity	4	1	3	6	-	13
Deployment	-	2	3	2	5	12
Total	18	17	37	25	17	114

Table 2. Process-based Estimation Table

With Process-based estimation, it deduced that the project will take an estimated 114 person-days, which translates to 38 person-days per person. This estimate is also feasible within the project timeline.

Cost Estimate

Item	Basis	Cost
Development Labor	~1300 hours x ₱0 per hour	₱0
Domain Registration	~₱1500 per year	₱1500
Server Hosting (Election Period)	~₱5000 per month x 5 months	₱25000
Total Cost		₱26500

Table 3. Cost Estimation Table

System Requirements

- **Application Server:** 2 vCPU, 2 GB RAM, PHP 8.5, Apache/Nginx (CodeIgniter MVC)
- **Database Server:** MySQL, 5 GB storage for election data
- **Network and Bandwidth:** 1 Gbps uplink
- **Client Requirements:** Modern web browser on desktop or mobile
- **External Services:** UP Mail for authentication, CRS for voter block data

Schedules and Milestones

The team will be working on the project throughout the semester. Given below is the GANTT chart for Halalan:

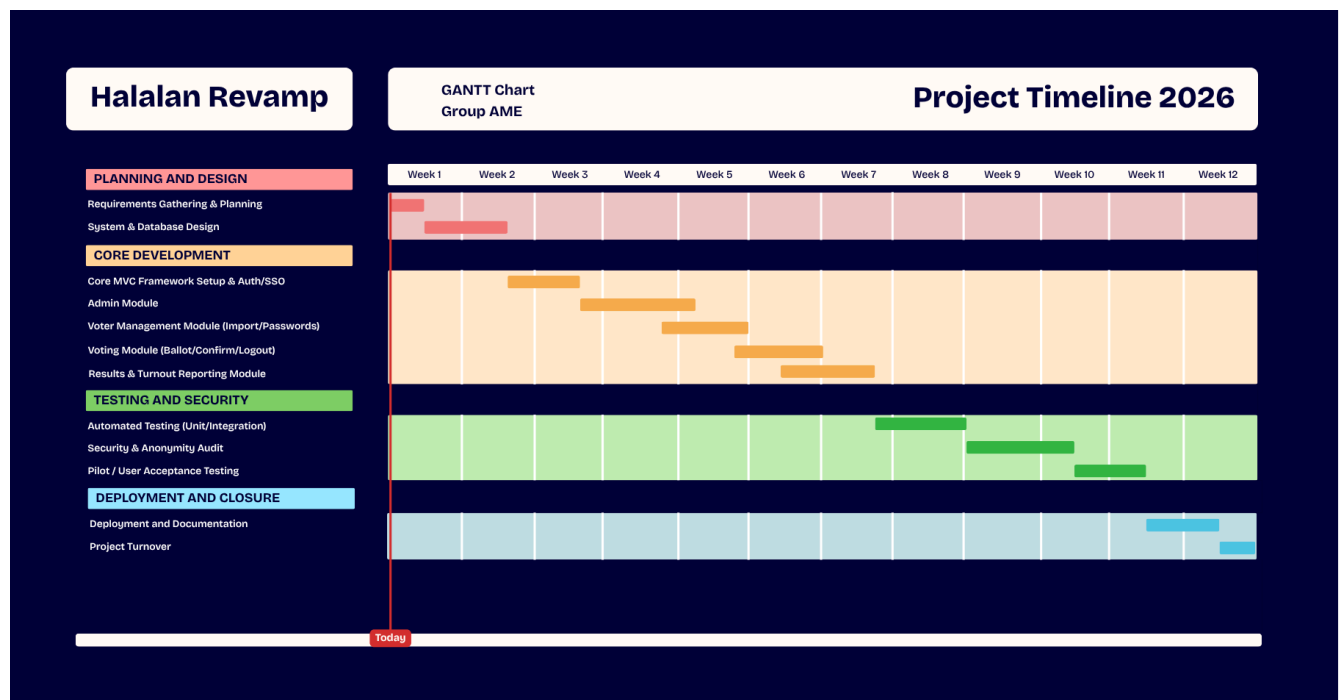


Figure 6. Project GANTT Chart

Project metrics

To assess the progress, code quality, and overall health of the Halalan revamp project, the following metrics will be tracked:

- **Time elapsed for a process (t_{queue}):** Measures schedule variance and task completion times across development cycles.

- **Difficulty assessment by team members, i.e. effort realized and estimated (W_{eval}):** Evaluates actual effort expended against initial estimations for high-complexity features.
- **Errors uncovered during work to make change (E):** Internal errors identified through code reviews, unit testing, and automated test pipelines prior to release.
- **Defects uncovered after change is released to the customer base (D):** External defects reported during live campus election trials or end-user testing.

Effort breakdown

Project Risks

The possible risks for the project are displayed below in Table 3 based on Chapter 28 of Pressman (2010).

PROJECT RISKS			
RISKS	PROBABILITY	IMPACT	RMMM
Unfamiliarity with chosen language and frameworks	90%	Marginal	Allocate time in learning the current architecture building upon existing foundational knowledge
Live testing reveals security vulnerabilities	60%	Critical	Follow secure coding practices and research vulnerabilities of

PROJECT RISKS			
			similar platforms
Test cases not exhaustive enough and fail to reproduce live environment	40%	Catastrophic	Use previous data to simulate real user volume
Revamped version not available for next election cycle	30%	Critical	Fallback to the current deployed version.
Framework/library used loses active support.	50%	Critical	Proper documentation to easily understand the codebase in case of migrating to supported frameworks

Table 4. Risk Table

References

Pressman, R. S., & Maxim, B. R. (2010). *Software engineering: A practitioner's approach* (7th ed.). R.S. Pressman & Associates, Inc.